Ex.No:1
26/7/24

Basic Networking Commands

Aim:

To study the various network commands used in Linux and windows.

(a) Commands in windows:

(i) arp -a: ARP - Address Resolution Protocol.

It shows the IP addresses and their corresponding MAC address of devices connected to the network. IP address - address given by the router to find which network it's using. It's like name. MAC address - It's a permanent one given by manufacturer arp-a shows the devices that my computer has recently communicated with. These entries are stored in ARP cache.

(ii) hostname: It's used to display the name of the computer

(iii) ipconfig /all: used to display complete network configuration details of a computer. It displays: Hostname, MAC address, IP Address, Default Gateway [entry to the outside network], DHCP [assigns the IP address], DNS [converts the web server name to IP address].

(iv) nbtstat -a: Used to display the NetBIOS name information of a computer & used to troubleshoot NetBIOS name resolution problem.

(v) netstat: Used to view the active network connections of my computer. It helps us to know which devices or servers my computer is communicating with.

(vi) nslookup: Used to perform DNS lookups and find the IP address of a website or domain name.

(vii) pathping: Used to trace the path between my computer and a destination. It also checks each router on the path for packet loss and helps identify where network problems occur. combines ping & tracert.
ping - Can I reach the destination.
tracert - which path did I take?

(viii) Ping: Used to check whether another device or website is reachable over the network. It sends ICMP Echo Request packets & waits for Echo Reply packets to verify the connection between two devices.

(ix) Route: Used to display & manage the routing table of a computer. It helps configure static routes so that data packets travel through the correct path to reach the destination.

(b) Commands in Linux:

(i) ip:

Used to view & manage IP address, network interfaces and routing information.

(ii) ifconfig: Used to configure & troubleshoot network interfaces. It has now been replaced by the ip command.

(iii) mtr:

Matt's Traceroute. combines ping & traceroute.

(iv) tcpdump:

Capture tool used to capture and display network packets for network monitoring and troubleshooting.

(v) ping:

whether the destination host is reachable or not over the network.

Working:

(i) ip <option> <object> <command>

Uses:

->ip address show - shows ip address on server.

→ ip addres add 192.168.1.254/24 dev enps 03 - this is used to assign ip addres to interface.

→ ip addres del 192.168.1.254/24 dev-enps 03 - used to delete an ip addres in the interface.

→ ip link set eth0 up - alter status at interface by bringing interface eth0.

→ ip link set eth0 daon - alter the status of the interface eth0.

→ ip route default add via 192.168.1.254 dev eth0 - add default route to the gateway.

→ ip route get - display route taken for the ip.

(ii) mtr <options> hostname/ip

→ mtr google.com - basic command that shows all details.

→ mtr -g google.com - gives all numeric IP addresses instead of hostnames

→ mtr -b google.com - shows numeric IP addresses and hostname.

→ mtr -c 10 - used to set the number of pings you want to.

(iii) tcpdump - used for capturing packets.

→ dnf install -y tcpdump - installing the command.

→ tcpdump -D - This allows users to filter packets based on hosts, protocol, port etc..

→ tcpdump -i eth0 - This is used to capture traffic in eth0.

→ tcpdump -i eth0 -c 10 host 8.8.8.8 - this is used to filter traffic from host 8.8.8.8

→ tcpdump -i eth0 src host 8.8.8.8 - this is used to capture traffic from 8.8.8.8

→ tcpdump -i eth0 dst host 8.8.8.8 - this is used to capture outbound traffic going to 8.8.8.8

→ tcpdump -i eth0 port 53 - this is used to capture only DNS port 53 traffic

→ tcpdump -i eth0 host 8.8.8.8 and port 53 - this is used to capture 8.8.8.8 hosts traffic from port 53.

→ tcpdump -i eth0 -c 10 host www.google.com and port 443 - this is used to capture HTTPS traffic.

→ tcpdump -i eth0 port not 53 and not 25 - this is used to capture all port traffic except port 80 and 25.

(iv) ping google.com

o/p: PING google.com (216.58.206.174) bytes of data.

64 bytes from iso fo 2927 - in f14le100.1
(216.58.206.174): icmp - seq = 1 ttl = 56
time = 10.7 ms.
:

(v) ping -c 10 google.com - this returns the host pinged 10 times.

(vi) nmcli - If you want to connect a host to network over ethernet, the connection's setting on command line using this command.

→ nmcli connection show : Lists the network manager connection Profiles

→ nmcli connection add con-name <connection-name> ifname <device-name> type ethernet - this is used to add the connection.

→ nmcli connection modify "wired Connection" - this is used to rename connection profile.

Result :

Thus the basic networking commands in windows and linux were studied.

3/8/20

Ex.No: 2
11/8/24

Study Of Network Cables

Aim:

To study different network cables.

a) Understand different types of network cables.

There are four major type of cables used in networking.

→ Unshielded twisted pair (UTP) cable.
→ shielded twisting pair (STP) cable
→ coaxial cable.
→ fibre optic cable.

Cable type	Category	Maximum Data transmission	Advantages Disadvantages	Applications
UTP	Category 3	10 bps	Advantages: ⇒ cheaper in cost ⇒ easy to install as they have a small ~~distance~~ diameter. Disadvantages: ⇒ more prone to EMI	10 Base-T-Ethernet
	Category 5	Up to 100 mbps		Fast ethernet gigabit ethernet
	Category 5e	1 Gbps		Faster ethernet gigabit ethernet.

Cable type	Category	Maximum Data transmission	Advantages Disadvantages	Applications
STP	Category 6, 6a	10 Gbps	Advantages ⇒ shielded ⇒ faster ⇒ less susceptible to noise. Disadvantages ⇒ expensive ⇒ greater installation effort	gigabit ethernet 55m widely used in data centers
SSTP	Category 7	10 Gbps		
Coaxial cable	RG-6 RG-59 RG-11	10-100 Mbps	Advantages ⇒ less bandwidth ⇒ high bandwidth ⇒ versatile ⇒ immune to interference Disadvantage ⇒ limited distance ⇒ cost ⇒ size is bulky	speed of signal is 50m Television network. high speed internet connections

Cable type	Category	Maximum Data transmission	Advantages Disadvantages	Applications
Fibre optics cable	Single mode Multi mode	100 Gbps	Advantages → high speed → high bandwidth → security Disadvantages → expensive → requires skilled installers	Maximum distance of fibre optics cable is around 100 meters.

Result:-

Thus the types of cables were studied.

8/8/24

3/10/24 Ex.No:3

INTRODUCTION TO CISCO PACKET TRACER

Aim:

To study the installation and user interface of cisco packet tracer and analyse the behavior of hub and switch.

Introduction:

Cisco Packet Tracer is a network simulation tool used to design, configure and troubleshoot networks without physical hardware. It is available on windows and linux and simulates real networking devices & protocols.

Installation:

→ download packet tracer from cisco Networking academy.

→ install it by following setup wizard

→ Launch application after installation

Procedure:

* Create two networks
  - → 4 PCs connected to Hub
  - → 2 PCs connected to switch

* Connect devices using copper straight through cables.

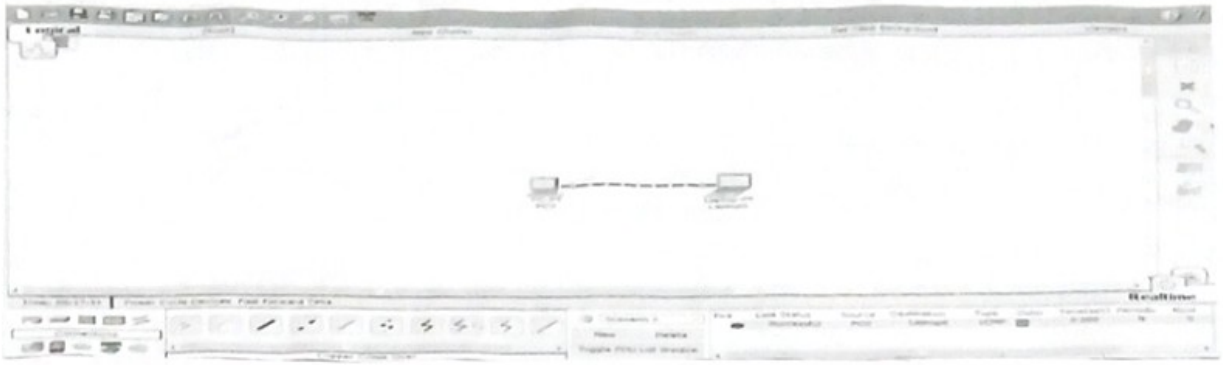
* Assign IP addresses to all PCs
* Send a PDU between 2 PCs
* Observe packet flow in realtime / simulation mode.
* Compare the working of switch & Hub.

Observation:

Hub: Broadcasts data to all connected devices, causing unnecessary traffic.

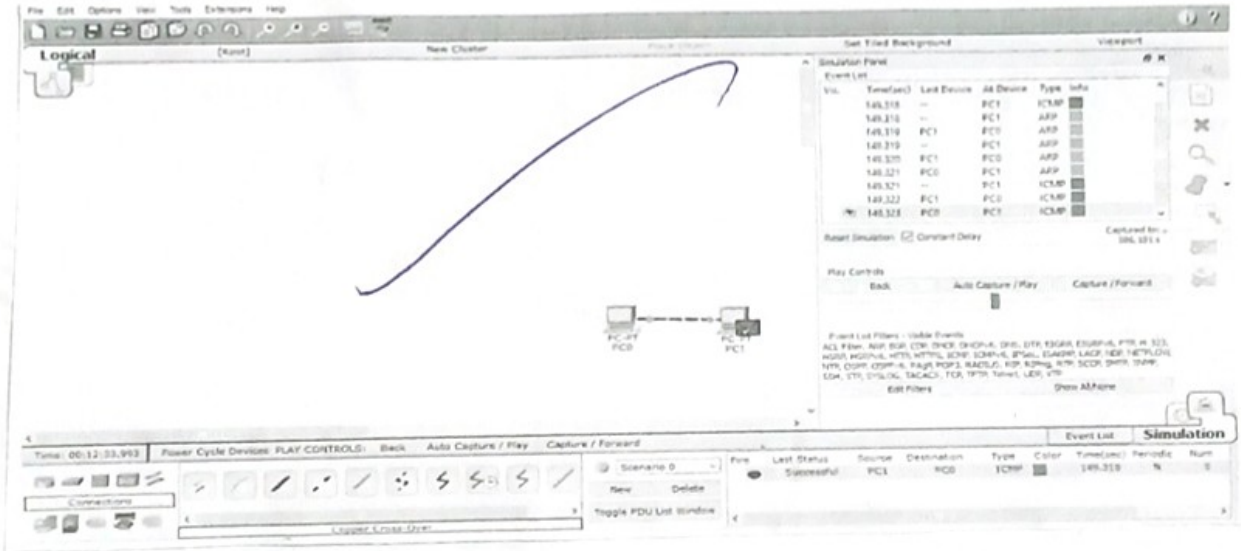
Switch: Sends data only to the destination device using MAC addresses, making communication faster and more efficient.

PEER - PEER Network, PC to Laptop



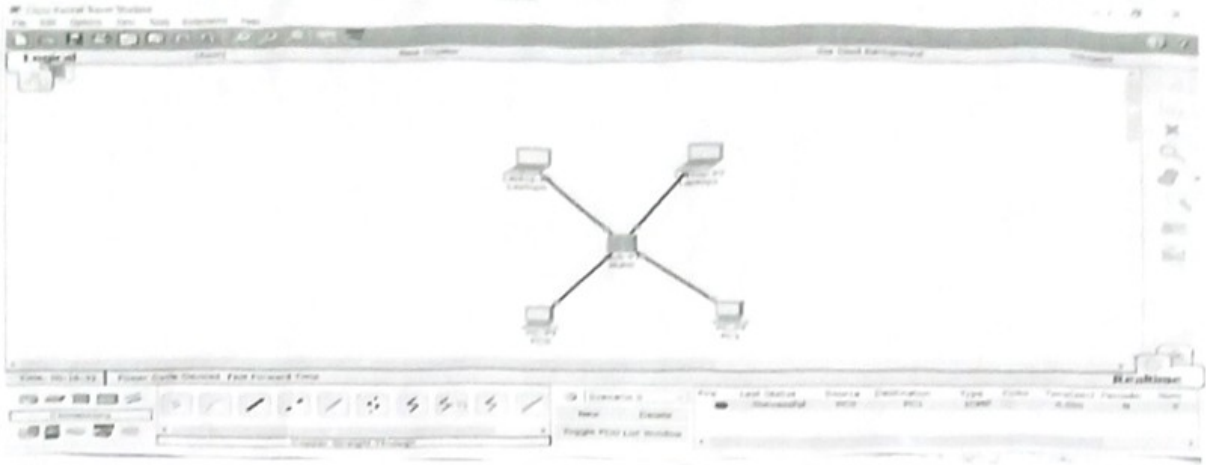
→ Connects PC to laptop directly to share data.

Peer - Peer Network : PC to PC



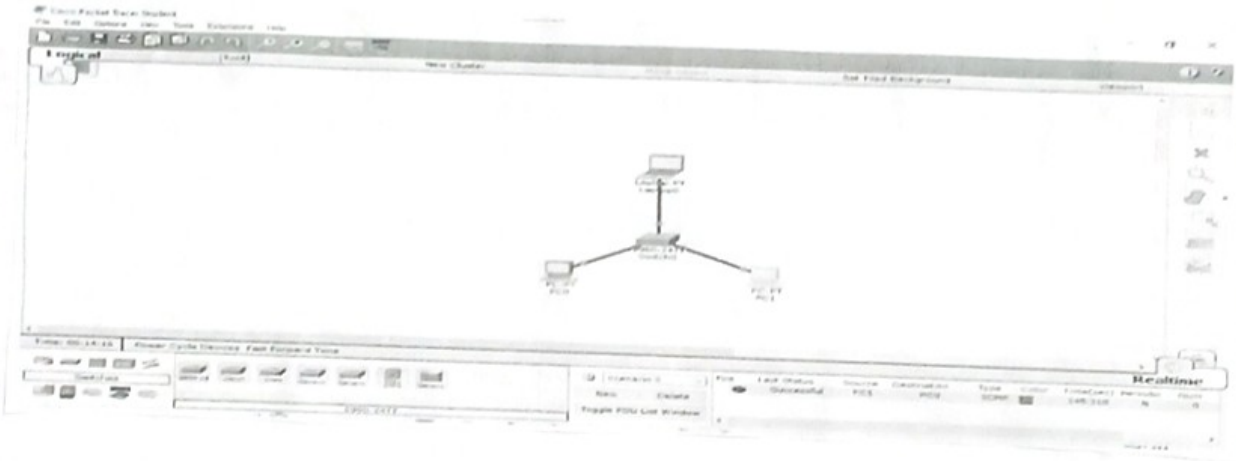
→ connects 2 PCs to share data directly.

Hub Network:



→ Broadcasts data to all devices that are available.

SWITCH NETWORK:



→ sends data only to destination device. using MAC addresses.

→ Makes communication faster.

Result:

Thus the experiment was carried out successfully.

3/8/26

Exp: 4
10/8/26

SETUP AND CONFIGURE A LAN USING SWITCH AND ETHERNET CABLES

AIM: To setup and configure a LAN using switch and ethernet cables.

What is LAN?

LAN stands for local Area Network which is a group of computers and connected devices linked together in a small physical location like a home, school, or office building.

How to setup LAN

Step 1: Connect the Hardware

⇒ place your router in a central location near a power outlet.

⇒ Connect your modem to the router's WAN or Internet port using an Ethernet cable

⇒ plug the router into power and wait for it to turn on.

Step 2: Connect your devices.

→ Wired setup :- plug one end of an ethernet cable into your computer's network port and the other end into one of the numbered LAN ports on the router.

Step 3 :- Configure the router

→ Type your IP address (192.168.1.1 or (192.168.0.1) into the address bar and press enter.

→ Log in using the default username and password printed on the router.

Step 4 :- Check the connectivity between switch and other machine by using ping command in the command prompt of the device.

ex:- ping 192.168.0.1

Step 5:- Try to access the shared folder from other computers of the network.

Result:-

Wireless LAN using switch and ethernet cables are executed successfully in cisco packet tracer.

10/8/24

19/8/26

Exp: 5

WIRESHARK

Aim :-

Experiments on packet capture tool: wireshark.

To filter, capture, view, packets in wireshark Tool. Capture 100¢ packets from the Ethernet : IEEE 802.3 LAN interface & Save it.

Procedure :-

* Select Local Area connection in wiresharks
* To capture option.
* Select stop capture automatically after 100 packets.
* Then click start capture
* To search TCP packets in search bar.
* To see flow graph click statistics flow graph.
* Save the packets.
*

1. Create a Filter to display only TCP/UDP packets, inspects the packets and provide the flow graph.

* Select LAN connection in wiresharks.
* To capture option.
* Select stop capture automatically after 100 packets.
* Then click start capture
* Save the packets.

2. Create a filter to display only ARP packets and inspect the packets.

Procedure :-

* Go to capture option
* Select stop capture automatically after 100 packets.
* Then click start capture.
* Search ARP packets in search box.
* Save the packets.

3. Create a filter to display only HTTP packets and inspect the packets procedure.

* Select Local Area connection in wireshark
* Go to capture option.

* Select stop capture automatically after 100 packets

* Then click start capture

* Search HTTP packets in search bar.

4. Create a file to display only DNS packets and provide the flow graph.

Procedure:-

* Go to capture options

* Select stop capture automatically after 100 packets.

* Then click start capture

* search DNS packets in search bar.

* To see the flow graph click statistics flow graph.

5. Create a filter to display only DHCP packets and inspect the packets.

* Select local Area connection in wireshark.

* Go to capture options

* Then click Select stop capture automatically after 100 packets.

* Then click start capture.

* Search DHCP packets in search bar.

* Save the packets.

Result:-

Wireshark on packet capture tool experiment are learned and executed successfully.

19/8/25

7/09/26

Exp 6 :: Error Detection & Correction Using Hamming Code

Aim:

Write a program to implement error detection and correction using hamming code concept and a test run to input data stream and verify error correction feature.

Sender:

* Input to sender file should be a text of one length. program should be convert the text to binary.

* Apply hamming code concept on the binary data and add redundant bits to it.

* Save this output in a file called channel.

Code:

```
data = input ("Enter 4 bits:")
```

```
d1, d2, d3, d4 = map(int, data)
    P1 = d1 ^ d2 ^ d4
    P2 = d1 ^ d3 ^ d4
    P4 = d2 ^ d3 ^ d4
code = [P1, P2, d1, P4, d2, d3, d4]
Print("sender Hamming code:", "".join
        (map(str, code))).
```

Receiver side :-

* Receive the hamming code from the sender.
* Enter the received code.
* Calculate the parity check values.
* Find the error position using the parity values.
* Correct the error by changing the bit.
* Display the corrected hamming code.

Code :-

```
r = list(map(int, input("Enter received 7
                        bits:")))
r1 = r[0] ^ r[2] ^ r[4] ^ r[6]
```

```
r2 = r[1] ^ r[2] ^ r[5] ^ r[6]

r3 = r[3] ^ r[4] ^ r[5] ^ r[6]

error = r1 * 1 + r2 * 2 + r4 * 4

if error == 0:
        print("No Error")
else:
        print("Error at position:", error)
        r[error - 1] ^= 1
        print("corrected code:", "".join(map(
                str, received)))
```

Input:

Enter 4 data bits: 1011

Sender Hamming code: 0110011

Received 7 bits: 0010011

Output:

Error at position: 2

Corrected code: 0110011

Sample Input 2:

Enter 4 data bits: 0010

Sender Hamming code: 0101010

Enter received 7 bits: 0101010

Output:

-NO Error-

Result:

Error detection & Error correction using hamming code was executed successfully.

7/9/20

22/9/24

Exp 7: NMAP to Discover live Hosts Using Nmap Scans on the TryHackme Platform

Aim:

To Discover live Hosts using Nmap Scans (ARP, ICMP, TCP/UDP) on the TryHackme platform Room link; https://tryhackme.com/room/nmap

Procedure:

Task 1 :- Introduction

* ARP Scan: This scan uses ARP requests to discover live hosts.

* ICMP scan: This scan uses ICMP requests to identify live hosts.

* TCP/UDP Ping scan: This scan sends packets to TCP ports and UDP ports to determine live hosts.

Task 2 :- Subnetworks.

Network segment :- It is a collection of computers linked toget

through a common "medium".

Subnetworks - It is usually the equivalent of one or more networks/segments linked together and configured to utilize the same router in an IP network.

* Subnets with /16
* Subnets with /24

Task 2 :- Enumerating Targets

We need to identify the targets we want to scan before we conduct any action.

1) List
- <MACHINE-IP> <scanme.nmap.org> <example> will scan 3 IP addresses.

2) Range
- 10.11.12.15-20
- will scan 6 IP addresses

3) Subnet
- <MACHINE-IP>/30
- will scan 4 IP addresses

nmap -sL <TARGETS>

Task-4 :- Discovering live Hosts.

1) ARP from link layer - commonly

used for Man-In-Middle-Attack (ARP spoofing and ARP poisoning).

2) ICMP from network layer - commonly used in DDOS attacks.

3) TCP from Transport layer - SYN flooding, TCP reset, TCP sessions hijacking attacks.

4) UDP from Transport layer - Also used in DDOS attacks.

Task 5 :- Nmap Host Discovery using ARP.

1) When a privileged user tries to scan LAN (Local area Network), Nmap uses ARP requests.

2) to discover internet hosts without port scanning - live systems use nmap -sn. Targets.

3) nmap -PR -Sn <TARGETS>

-PR specifies that you only want an ARP scan.

4) nmap -PR -Sn <MACHINE IP>/24

5) nmap -PR -Sn <ATTACKBOX - IP>/24

Task 6 :- Nmap host discovery using ICMP

1) Add -PE option to utilize ICMP-echo requests to discover live hosts.

2) nmap -PE -sn TARGET to check host is live.

3) -PP option instructs Nmap to use ICMP timestamp requests.

nmap -PP -sn TARGET

4) - PM option can enable this scan to ICMP address mast requests.

nmap -PM -Sn TARGET

Task 7 :- Nmap Host discovery using TCP and UDP.

1) Nmap to use TCP SYN Ping use -Ps followed by the port number, range, list or a combination of this options.

-PS21, -PS25 targets ports 21, 22, 23, 24, 25.

2) nmap -PS -sn TARGET ~~ports 21, 22, 23~~ check the TCP port is open.

3) TCP ACK ping - you must be running

Nmap as a privileged user.

4) nmap - PA - sn TARGET to discover online hosts on the target's subnet.

5) UDP ping - sending a UDP packet to and open port is unlikely to result in a response.

6) If we transmit a UDP packet to a closed UDP port, we should expect to receive an ICMP port unreachable packet.

nmap -PU - sn TARGET.

Task 8:. Using Reverse - DNS lookup

1) Nmap is to use reverse - DNS internet hosts.

2) -R option to query the DNS server even for offline hosts.

Task 9:- Summary:

ARP scan, ICMP Echo scan, ICMP Timestamp scan, ICMP address mark scan, TCP, SYN Ping scan, TCP ACK ping scan, UDP Ping scan.

Result:

NMAP to discover live host using tryhackme platform was successfully executed.

22/11

28/9/24

Exp 8 :- End to End Communication at Transport Layer

a)

Aim :- Implement echo client server using TCP / UDP packets.

Procedure :-

* Create a socket using python socket programming.
* Specify the IP address and port number.
* Bind the server socket to the IP address and port.
* Make the server listen for client connections
* Connect the client to the server
* Send and receive message between client & server.
* Display the received message
* Close the socket connection.

Code :-

TCP - server

```
import socket
server = socket.socket(socket.
AF_INET, socket.sock_STREAM)
```

```
server.bind ("127.0.0.1", 5000))
server.listen(1)
print ("server is waiting for connection.")
conn, addr = server.accept()
print ("client connected")
while True:
    data = conn.recv(1024).decode()
    if not data:
        break
    print("client:", data)
    conn.send(data.encode())
conn.close()
server.close()
```

```
TCP_Receiver(client):
    import socket
        client = socket.socket(socket.
    Socket.SOCK_STREAM)        AF_INET,
        client.connect(("127.0.0.1", 5000))
    message = input("Enter message:")
    client.send(message.encode())
    reply = client.recv(1024).decode()
    print("server:", reply)
    client.close()
```

Terminal 1 - server

Python tcp_server.py

server is waiting for connection...

client connected

client : Hello

Terminal 2 - client

python tcp_client.py

Enter message : Hello

Server : Hello

Tcp echo is working.

Result :-

The TCP/UDP packets using echo client server socket program is executed successfully.

28/9/20

b)

Aim: Implement chat client server using TCP packets.

Code:

Chat - server side:

```
import socket
    Server = socket.socket(socket.AF_INET,
socket.SOCKET_STREAM)
server.bind ("127.0.0.1",6000)
server.listen(1)
print("Waiting for client...")
conn, addr = server.accept()
print ("client connected")
while True:
    message = conn.recv(1024).decode()
    if message.lower() != "bye":
        print("client ended chat")
        break.
    print("client:", message)
    reply = input("server:")
    conn.send(reply.encode())
if reply.lower() == "bye":
conn.close()
server.close()
```

chat - client side

```
import socket
client = socket.socket(socket.AF_INET,
socket.SOCK_STREAM)
client.connect(("127.0.0.1", 6000))
print("connected to server")
while True:
    message = input("client:")
    client.send(message.encode())
    if message.lower() == "bye":
        break
    reply = client.recv(1024).decode()
    print("server:", reply)
    if reply.lower() == "bye":
        break
client.close()
```

Terminal 1

```
python chat_server.py
waiting for client....
client connected
client: Hello
server: Hi'
client: How are you.
server: Fine
```

Terminal 2

```
python chat_client.py
connected to server
client: Hello
server: Hi'
client: How are you?
server: Fine.
```

Result:

The chat programming using sockets by TCP client & server was implemented successfully.

28/9/25

Exp 9
29/9/22

IMPLEMENT YOUR OWN PING PROGRAM

Aim:-

To write and execute a python program to implement ping using UDP client-server communication & calculate the round trip time (RTT) for the packets sent between the client and server.

Program :-

```
import socket
import time
import sys
# UDP ping server
def udp_ping_server(host="127.0.0.1", port=12000):
    server_socket = socket.socket(socket.
        AF_INET, socket.SOCK_DGRAM)
    server_socket.bind((host, port))
    print(f"[server] listening on {host}:{port}")
    while True:
        msg, client_address = server_socket.
            recvfrom(1024)
        print(f"[server] received '{msg.
decode()}' from {client_address}")
While True:
```

```
msg, client_addres = server_socket.
        recv from(1024)

# UDP ping client

def
  udp_ping_client(server_host = "127.0.0.1",
  server_port = 12000, count = 5):
        client_socket = socket.socket[socket.
    AF_INET, socket.SOCK_DGRAM)
  client_socket.settimeout(1)
  for i in range(1, count + 1):
        msg = f'Ping {i} {time.time()}'
        start = time.time()
        client_socket.sendto(msg.encode(),
        (server_host, server_port)

  try:
        data, _ = client_socket.
  recv from (1024)
                end = time.time()
                rtt = (end_start) * 1000
                print(f"reply from {
    server_host} {server_port} |"
  f"{data.decode()} | RTT = {rtt.2f}
                ms")
        except socket.timeout:
                print()
```

```
        child_socket.close()

# run as server or client
if __name__ == '__main__':
    if len(sys.argv) > 1 & sys.argv[1] == "server":
        udp_ping_server()
    else:
        udp_ping_client()
```

Output:

Server side

```
[server] Listening on
127.0.0.1:12000
[server] Received 'Ping' 1728123456
from ('127.0.0.1', 54321)
...
```

Client side

```
Reply from 127.0.0.1:12000 | Ping
1728123456.123 | RTT = 0.32 ms.
```

Result:

Thus, the UDP ping client-server program sucessfully executed. The Ping message were transmitted & echoed back sucessfully.

29/9/25

on 19/24
Exp 10

FLOW CONTROL AT DATA LINK LAYER

Aim:

To implement flow control at the Data Link Layer using the sliding window Protocol and simulate the flow of frames from one node to another.

Sliding Window Protocol:

Flow control technique used to control the number of frames sent before receiving acknowledgements. It allows the sender to transmit multiple frames within a fixed window size without waiting for an acknowledgement after every time.

Requirements:

* Input the window size
* Input a text message.
* Consider one character as one frame.

* Each frame contains frame number & data.
* Send frames & wait for acknowledgement.
* Use ack for correctly received frames.
* Use NACK for an incorrect frame is received.

Working:

Sender:

1. Enter a window size
2. Enter the message.
3. Divide the message into frames.
4. Send frames according to the window-size.
5. Store the frames in sender-buffer
6. Wait for acknowledgement.
7. If the expected ACK is received, send the next set of frames.
8. If nack is received, resend the required frame(s)

Receiver:

1. Read frames from sender-buffer.

2. check the frame no.
3. If the frame no. is correct, send ack.
4. If the frame no. is incorrect, send NACK through receiver-Buffer.

Code:- (Python)

```
import time
window = int(input("Enter window size:"))
message = input("Enter message:")
start = 0
while start < len(message):
    print("SEnder")
    end = start + window
    if end > len(message):
        end = len(message)
    sender_buffer = []
    for i in range(start, end):
        frame = (i, message[i])
        sender_buffer.append(frame)
    print("Frame No:", i, "Data:", message)
print("In sender Buffer:")
print(sender_buffer)
print("In waiting for receiver...")
time.sleep(1)
print("Receiver")
expected = start
error = false
for frame in sender_buffer:
    frame_no = frame[0]
    data = frame[1]
```

```
print ("Received Frame:", frame_no,
       "Data", data)

if frame_no == expected:
        expected = expected + 1
else:
        error = True
        break

if error:
        received_buffer = ("NACK", expected)

print("Transmission Completed")
```

Input:

Enter Window size : 4
Enter message : HELLO

Outuput:

Sender

Frame no : 0 Data : H
Frame no : 1 Data : E
Frame no : 2 Data : L
Frame no : 3 Data : L

Sender Buffer :

[(0, 'H'), (1, 'E'), (2, 'L'), (3, 'L')]

Waiting for receiver ...

Receiver

received Frame : 0 Data : H

received Frame : 1 Data : E

received Frame : 2 Data : L

Received Frame : 3 Data : L

Receiver Buffer

('Ack', 3)

Sender checking Response

ACK received : 3

Transmission Completed .

Result :

Thus, flow control using the sliding window protocol was successfully implemented & the transmission of frames, ACK & NACK was verified.

21/9/20

Exp. 11

RAW SOCKET PROGRAMMING TO IMPLEMENT PACKET SNIFFING

Aim:

To write and execute a python program using raw sockets to implement packet sniffing and display the source IP address, destination IP address, protocol and packet size of captured IP packets.

Program:

```
import socket
import struct

HOST =
  socket.gethostbyname(socket.gethostname
Sock = socket.socket(
        socket.AF_INET,
        socket.SOCK_RAW,
        socket.IPPROTO_IP
)

Sock.bind((HOST,0))
Sock.setsockopt(socket.IPPROTO_IP,
  socket.IP_HDRINCL,1)
Sock.ioctl(socket.SIO_RCVALL,
  socket.RCVALL_ON)
```

```
print("Packet sniffing started...")
print("HOST IP:", HOST)
print("Press Ctrl + C to stop.\n")

try:
    while True:
        packet, address = sock.recvfrom(65535)
        ip_header = packet[0:20]
        iph = struct.unpack("!BBHHHBBH4s4s"
    ip_header)
        source_ip = socket.inet_ntoa(iph[8])
    destination_ip = socket.inet_ntoa(iph[9])
        protocol = iph[6]

    print("=========================")
        print("Source IP:", source_ip)
        print("Destination IP:", destination_ip)
        print("Protocol:", protocol)
        print("Packet size:", len(packet))
    print("=========================")
except keyboard interrupt:
        print("\n Sniffing stopped.")
finally:
    sock.ioctl(socket.SIO_RCVALL, socket.
            RCVALL_OFF)
    Sock.close()
```

Output:

```
Packet sniffing started...
Host IP: 192.168.1.10

Press Ctrl+C to stop.
==========
Source IP: 192.168.1.10
Destination IP: 142.250.183.14
Protocol: 6
Packet Size: 60
==========
Source IP: 142.250.183.14
Destination IP: 192.168.1.10
Protocol: 6
Packet Size: 52
==========
Sniffing stopped.
```

Result:

Thus, the python program using raw sockets was executed to implement packet sniffing and display the source IP, destination IP, protocol and packet size of captured IP Packets.

12/10/24

Exp.12
21/10/26

Subnetting in Cisco Packet Tracer

Aim:

To implement classless IP subnetting using Cisco packet Tracer and verify the connectivity between different subnets.

Requirements:

* Cisco packet Tracer
* Routers
* switches
* PCs
* Ethernet cables.

Procedure:

1. Open Cisco packet Tracer
2. Create a new network topology
3. Add routers, switches, and PCs to the workspace.

Connect the device using ethernet Cables.

4. Divide the network into smaller Subnets using the /27 subnet mask.

5. Configure the IP address and subnet mask of the router interfaces.

6. Configure the IP address, subnet mask and default gateway for each PC.

7. Configure the required switch ports.

8. Enable the router interfaces using the no shut down command.

9. Check the IP configuration of all devices and the ping command to check connectivity between PC.

IP - verify that the device in the network communication.

Result:-

Thus the implementing subnetting in the cisco packet Tracer was successful.

12/10/26